Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Систем Управления и Робототехники

Лабораторная работа №7

по Программированию

Вариант 48294

Выполнил:

Рязанов Даниил Дмитриевич

Группа R3140

Преподаватель:

Вальц Мартин Эдуардович

Санкт-Петербург 2025

Содержание

Текст задания *3*

Программная архитектура *3*

package core **3**

package client **3**

package server **4**

package collection **4**

package storage **4**

Диаграммы классов **4**

Server 4

Client 5

Упаковка и запуск программы *5*

Выводы по работе *6*

Текст задания

Доработать программу из лабораторной работы №6 следующим образом:

1. Организовать хранение коллекции в реляционной СУБД (PostgresQL). Убрать хранение коллекции в файле.
2. Для генерации поля id использовать средства базы данных (sequence).
3. Обновлять состояние коллекции в памяти только при успешном добавлении объекта в БД
4. Все команды получения данных должны работать с коллекцией в памяти, а не в БД
5. Организовать возможность регистрации и авторизации пользователей. У пользователя есть возможность указать пароль.
6. Пароли при хранении хэшировать алгоритмом `SHA-384`
7. Запретить выполнение команд не авторизованным пользователям.
8. При хранении объектов сохранять информацию о пользователе, который создал этот объект.
9. Пользователи должны иметь возможность просмотра всех объектов коллекции, но модифицировать могут только принадлежащие им.
10. Для идентификации пользователя отправлять логин и пароль с каждым запросом.

Необходимо реализовать многопоточную обработку запросов.

1. Для многопоточного чтения запросов использовать `Fixed thread pool`
2. Для многопотчной обработки полученного запроса использовать `Cached thread pool`
3. Для многопоточной отправки ответа использовать `Fixed thread pool`
4. Для синхронизации доступа к коллекции использовать `синхронизацию чтения и записи с помощью java.util.concurrent.locks.ReadWriteLock`

Программная архитектура

Все исходники программы расположены в Github-репозитории MrRyabena/DragonsDB.

package core

К существующим классам добавлен WireFrame – транспортная оболочка для передачи Request и Response по UDP с метаданными запроса и сессии.

package client

Архитектура клиента была незначительно изменена для запроса пароля у пользователя в форме скрытых символов.

package server

Архитектура сервера построена на конвейере обработки запросов. Все модули последовательно работают с объектом ServerContext, что упрощает расширение и изменение логики сервера.

В обработку добавлены отдельные классы для регистрации/входа пользователя.

⇓ **ConnectionHandler** – прием соединений;
⇓ **RequestReader** – чтение и декодирование запроса;
⇓ **AuthenticationHandler** – обработка регистрации и входа пользователя;
⇓ **CommandsHandler** – выполнение команд;
⇓ **ResponsePacker** – упаковка ответа;
⇓ **ResponseSender** – отправка результата клиенту;
⇓ **ClientRequestHandler** – объединения всех этапов обработки запроса.

Также сервер стал поддерживать сессии и многошаговые диалоги через SessionManager и ClientSession, а для авторизации используется отдельный AuthService.

package collection

Коллекция драконов была адаптирована под хранение с привязкой к владельцу. Для этого добавлен **DatabaseCollection**, который сохраняет и изменяет данные с учетом логина пользователя.

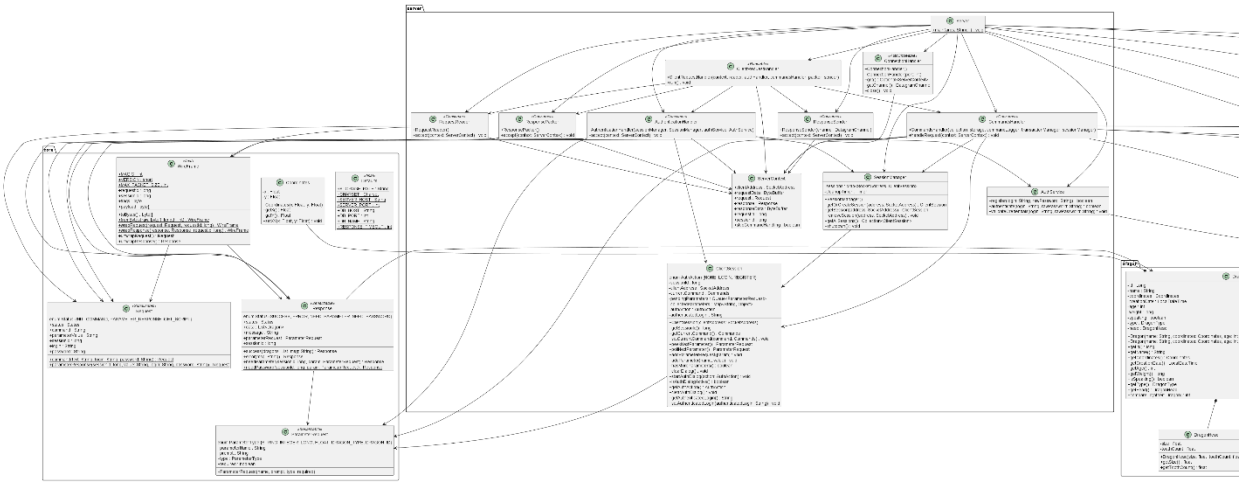
package storage

Слой хранения был переработан под PostgreSQL.

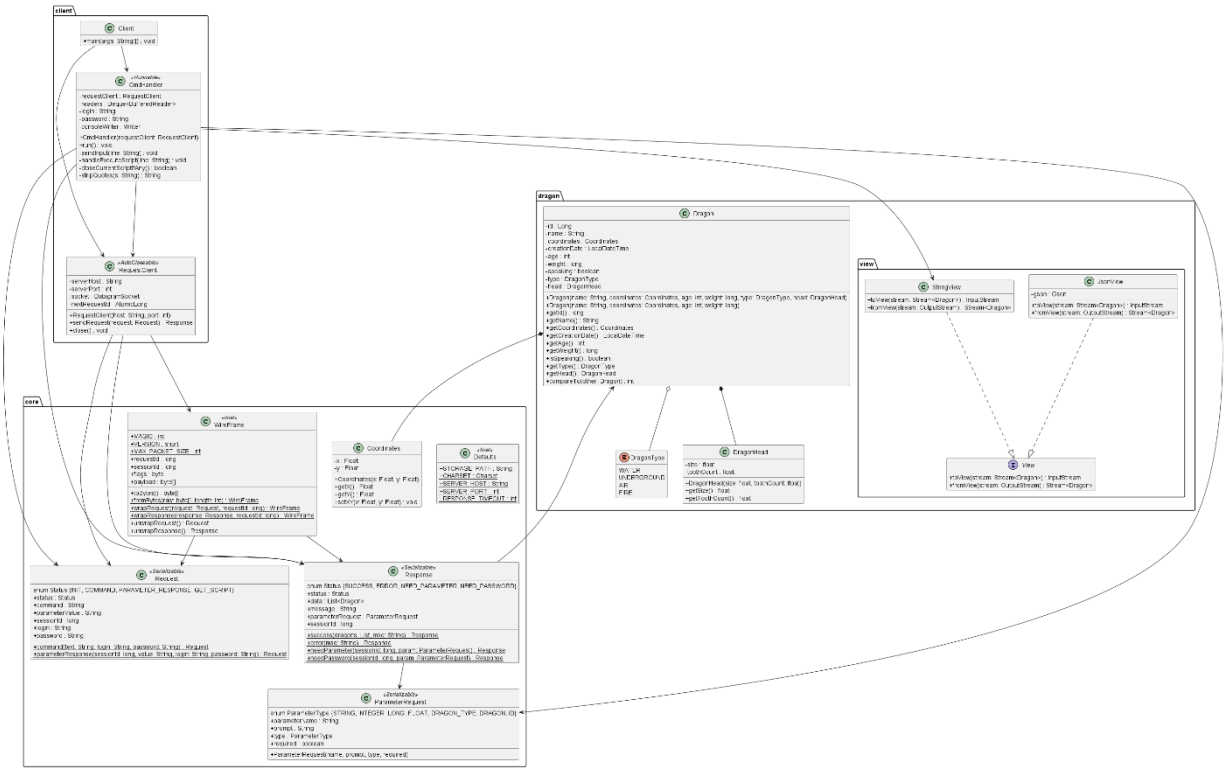
- **DatabaseStorage** отвечает за загрузку и сохранение данных;
- **PostgresSupport** содержит общие настройки подключения и создание схемы;
- **PostgresDragonRepository** выполняет операции чтения и записи в базе данных;
- **FileCommandLogger**, **TransactionManager** и **RecoveryManager** поддерживают журналирование и восстановление состояния.ч

Диаграммы классов

Server



Client



Упаковка и запуск программы

Исходный код программы расположен в папке src.
Для сборки, компиляции и упаковки программы используется утилита Maven. Сборка проекта разделена на 3 таргета:

- **common** – общий пакет для сервера и клиента;
- **server**
- **client**

Выводы по работе

В результате выполнения лабораторной работы приложение было существенно улучшено: архитектура стала более устойчивой, добавлена работа с базой данных и расширена серверная логика.
Практически я закрепил навыки разработки на Java, научился работать с PostgreSQL и лучше понял принципы многопоточности при построении клиент-серверных приложений.